Deep Learning Assignment 1

Ivar van der Spoel, Ivor Brouwer & Samuel van Erk

September 2024

Introduction

For assignment 1 of Deep Learning we are going to use the TensorFlow [AAB+15]/Keras[C+15] deep learning libraries, to implement neural networks that can perform classification and regression on image datasets. The goal is to learn how to create and train simple neural networks, gain practical experience by comparing neural network architectures and to get more intuition for hyperparameter tuning.

Task 1

For the first classification task, we use the MNIST-Fashion dataset [XRV17]. This dataset contains 28x28 pixel images of fashion items, associated with a label from ten classes. We develop two classification architectures, a multi-layer perceptron network (MLP) and a convolutional neural network (CNN). For both models, we experiment with different hyperparameters to get practical experience with tuning neural networks. To see if our tuned hyperparameters also perform well on another dataset, we use the best hyperparameters from the MNIST set to train models on the CIFAR-10 dataset [Kri09], which contains images from ten different classes such as airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks.

1.1 Methodology

1.1.1 MLP & CNN models

The MNIST fashion dataset consists of a training set (60,000 images) and a test set (10,000 images). We use 10% of the training set as a validation set. Let us first briefly explain both architecture types.

The MLP model starts by flattening the input images to a one-dimensional array. After the input layer comes a certain number of Dense layers with a dropout layer after each dense layer. The final layer is another Dense layer with a softmax activation function and ten output nodes: one for each class.

The CNN model starts with several convolutional (`Conv2D`) layers, each one followed by a two-dimensional Max Pooling (`MaxPooling2D`) layer. Subsequently, the data is flattened and passes through several Dense layers. Both the Max Pooling and the Dense layers are followed by a dropout layer. Once again, the final output layer is a Dense layer with ten output nodes.

1.1.2 *Ceteris paribus* search

To try and tune the hyperparameters, we start with a *ceteris paribus* approach, where one parameter is varied and all others are kept constant. We vary the number of (Dense and convolutional) layers, number of neurons, activation functions, weight initialization methods, optimizers, dropout rate, and learning rate. The default values for the unchanged parameters are shown in Table 1. Similar to the example implementations of CNNs from Keras, the number of neurons increases with each layer. We implement this by doubling the number of neurons with each layer.

Table 1: default values of unchanged parameters for the MLP and CNN models.

Parameter	**MLP default values**	**CNN default value**
# hidden layers	4	(3,2)
# neurons	128	32
Activation function	reLU	reLU
Weight initialization	He (normal)	He (normal)
Optimizer	SGD	SGD
Dropout	0.2	0.2
Learning rate	0.01	0.01
Kernel		(3,3)
Maxpoolsize		(2,2)

***Note*:** The parameters 'Kernel' and 'Maxpoolsize' are only relevant for CNN models. The number of hidden layers for the CNN model is denoted as the number of convolutional layers and dense layers respectively.

These default parameters were chosen by looking at the example implementations provided by Keras along with Table 10-2 from 'Hands-on Machine Learning with Scikit-Learn, Keras & TensorFlow' [Ge7]. Besides, the chosen number of epochs (20) and batch sizes (128) were also chosen by the common values used in these Keras examples.

1.1.3 Random search

This first approach is not very rigid, since it tries only a few combinations of different parameters. Therefore, we also perform a random search using the `RandomSearch` function from Keras Tuner. The results from the ceteris paribus search are used to narrow down the available parameters for the random search. The tuner tries 10 different combinations of hyperparameters for both models and saves the three best combinations for each.

1.1.4 Comparison with CIFAR-10

Finally, we use these best combinations of parameters to train models on the CIFAR-10 dataset, to see if our performance gains translate to another dataset.

1.2 Results

1.2.1 Ceteris paribus search

Notable findings are presented and discussed, providing information on different parameter tuning. All relevant Tables and Figures can be found in Appendix A.

Hidden layers
For the MLP model, the difference in test set accuracy (Table 6) is minimal for one to four hidden layers. Using six hidden layers, the model converges more slowly and with eight hidden layers the test set accuracy is significantly worse. Using two convolutional layers in the CNN model works much better than three convolutional layers. One dense layer converges quicker than two dense layers, but both result in comparable accuracy.

Number of neurons
For the MLP model, the test set accuracy mainly increases with the number of neurons. As seen in table 8, the CNN model shows a similar pattern, but using 128 neurons performs better than using 256 neurons. Still, using many neurons is not always desirable, since it also corresponds to a longer computation time and increases the risk of overfitting.

Dropout
On both models, as seen in Table 10, a dropout of 0 has the highest test accuracy. This suggests that there is little overfitting happening using the default parameters (Table 1). When using different hyperparameters, there is a risk of overfitting that can be mitigated by introducing a dropout. This makes using a dropout sensible, even if it means sometimes getting a lower accuracy.

Activation functions
In Table 9, it can be seen that for the MLP model, the Tanh and ReLU activation have a similar learning curve and test set accuracy (Figure 7). The model using a Sigmoid activation converges more slowly and has a lower test set accuracy. In the CNN model, we see again that Tanh and ReLU perform similar, but for this model using the Sigmoid function results in a very low test set accuracy and the model does not seem to learn until the last few epochs. Therefore, using a Sigmoid activation function does not seem useful.

Learning rates
In Figure 11, a learning rate of 0.01 is shown to be stable for both models. This is to be expected since the parameters are tuned very little after each epoch. This prevents overshooting, but it also results in a lower test set accuracy and a slower learning curve. For both models, the test set accuracy peaks at a learning rate of 0.1. The final losses and accuracies can be seen in Table 12.

Weight initialization methods
For the MLP model, all weight initialization methods perform similarly, as seen in Figure 7. For CNN, the Glorot normal initialization performs less well than both He initializations. It becomes obvious that, for this dataset specifically, the idea of Glorot initialization (and having issues with exploding gradients) is not necessary to improve

the model, it even worsened it.

Optimizers
Of the three optimizers that were tried, Stochastic Gradient Descent (SGD) is the only optimizer that works for the MLP model. For the CNN model, the Adam and RMSprop optimizer also seem to work, as seen in Table 11. RMSprop converges the quickest and SGD the slowest, however, the test set accuracy of all three initializations is similar.

CNN-specific parameters
We also varied the kernel size of the convolutional layer and the pool size of the Max Pooling layer, but these parameters showed little to no variation for different sizes on this specific dataset. However, as seen in Task 2, these parameters can become relevant, especially Max Pooling if larger images are used (see Section 2.3.2).

1.2.2 Random Search

The three best combinations of hyperparameters we found using the random search for both models are presented along with their test set accuracies in Table 2. Key takeaways are that, in all six best combinations, the ReLU activation and SGD optimizer are used as well as a Uniform or Normal He initialization. In other parameters, it is difficult to find a clear trend.

Table 2: The three best hyperparameter combinations for both architectures and the corresponding test set accuracy on the MNIST Fashion and CIFAR-10 dataset.

	MLP 1	MLP 2	MLP 3	CNN 1	CNN 2	CNN 3
# hidden layers	4	2	4	(2,1)	(2,2)	(2,2)
# neurons	256	256	512	64	64	256
Activation function	ReLU	ReLU	ReLU	ReLU	ReLU	ReLU
Weight initialization	He Normal	He Uniform	He Uniform	He Uniform	He Uniform	He Normal
Optimizer	SGD	SGD	SGD	SGD	SGD	SGD
Dropout	0	0	0.1	0.2	0	0.2
Learning rate	0.1	0.05	0.05	0.1	0.03	0.05
Kernel	-	-	-	(3,3)	(3,3)	(3,3)
Maxpoolsize	-	-	-	(2,2)	(2,2)	(2,2)
MNIST accuracy	0.885	0.890	0.884	0.915	0.908	0.907
CIFAR-10 accuracy	0.465	0.495	0.503	0.734	0.605	0.743

1.2.3 Comparison with CIFAR-10

The accuracy of the models trained on the CIFAR-10 dataset is presented in Table 2. It can be seen that the CIFAR-10 accuracy is significantly higher for the CNN architecture than for the MLP.

1.3 Discussion

Looking at Table 2, we can conclude that the CNN model is more accurate than the MLP model, and that the CNN model translates better to the CIFAR-10 dataset. This might have to do with the fact that CNN models utilize the two-dimensionality of the input data, whereas an MLP model immediately flattens the data to one dimension. We can also conclude best parameters depend a lot on the model architecture and the training/test set. Therefore, we likely need to tune the hyperparameters again in the following task.

An important parameter that we overlooked in our hyperparameter optimization is the batch size. Dominic Masters and Carlo Luschi (2018) [ML18] mention that a small batch size (< 32) often achieves the best training stability and generalization performance. We have used a larger constant batch size of 128. Adjusting this can increase the performance of our model. The number of epochs or iterations is also kept constant, at 20. According to Géron (2017) [Ge7], the number of iterations does not have to be tweaked, since this mainly influences the computation time, which can be avoided by using early stopping.

This task was mainly focused on getting acquainted with the influence of different hyperparameters and not as much on getting the highest accuracies. If we wanted to increase the accuracy of our models, we should focus on the CNN model, which performed better than the MLP model and we should perform a much more thorough random (or grid) search. A search through a greater hyperparameter space with more iterations (which was not possible for us due to computational limitations) could result in a higher accuracy.

Task 2

For Task 2 the goal is to find the correct time corresponding to an image of a clock (see Figure 1). This is done by training a CNN that can predict the time as correctly as possible. There are two datasets we will test on, which consist of 18,000 grayscale images with sizes 150x150 and 75x75. The labels for the clocks are in the format: (hours, minutes). We will create three different types of networks: a classification, a single-head regression, and a multi-head network.

Common sense error

The main goal of this assignment is to minimize the "common sense" error. When predicting the time we have to take the absolute value of the time difference between the predicted and the actual time, e.g. the difference between 11:30 and 0:30 is 1 hour not 11 hours. To account for the common sense error, the loss function must be more elaborate than a simple mean squared error function. We achieve this by calculating the absolute difference between the true value and the predicted value and then using the minimal distance between this difference and 12 - this difference. We will also look at label transformation to minimize this error.

Label transformation

Although not implemented, it is possible to reformulate the labels as periodic functions such as the sine and cosine functions. First, we have to change the labels to be two angles in the radians range [0,2π)] for the hours and the minutes. We would also need 4 outputs for this to work.

- $\text{hours}_x = \cos(\text{angle}_h)$
- $\text{hours}_y = \sin(\text{angle}_h)$
- $\text{minute}_x = \cos(\text{angle}_m)$
- $\text{minute}_y = \sin(\text{angle}_m)$

The unit circle ranges between [-1,1] in both the x and y direction, and the most common activation functions output range between [0,1]. So tanh would be the ideal activation function in this case, because, like these transformed labels, it also has an output range between [-1,1]. This not only allows the model to fit the periodic nature of the labels but also prevents outputs from drifting outside the desired range, improving training stability and accuracy.

2.1 Methodology

To find a model that performs well on the datasets we used the following method:

1. Create a base CNN.
2. Train model on the 75x75 dataset with an 80/10/10 split for train/validation/test sets.
3. With batches of 128 and in 30 epochs or less, try to reach greater than 75% accuracy or for regression a loss below 1.000 (hours).
4. Evaluate the model on the 150x150 dataset with an 80/20 split for train/test sets.
5. Report findings and perform comparison for the different models.

For each type of network we create different CNN models, as it appears that regressive and classification models require different types of features.

Figure 1: Example images of clocks and their corresponding labels to be predicted.

2.2 Results

Notable findings on the 75x75 dataset are presented and discussed, going over the different types of models and explaining how well they perform on the dataset.

2.2.1 Classification

To classify the time as an n-class classification problem, the labels were transformed to represent time intervals on the clock. First, classification was done for 24 categories, splitting up the time into half-hour intervals. An example of a category would be [3:00-3:30]. The implementation of the model can be found in Table 13.

The classification using 24 categories worked quite well, but as the number of categories increased the accuracy quickly decreased, as was expected. Below, in Table 3 this regression of the accuracy can be observed. 720 categories, classifying each minute for each hour, is impossible with this simple model.

Categories	Accuracy
24	83.00%
72	73.34%
144	63.67%
720	0.05%

Table 3: Accuracy per number of categories.

2.2.2 Single-head regression

For the single-head regression model, which predicts the time using a single output node, the labels are transformed to the following format: [03:00 $\rightarrow$ y = 3.0]; [05:30 $\rightarrow$ y = 5.5]. The implementation of the model can be found in Table 14.

One problem with this simple loss function is that it can cause the model to also output negative numbers, as we do not discourage this with this simple version. This is not a major issue as time is circular and we can fix this by post-processing the negative numbers by adding 12 to them. Plotting the prediction with the true value results in Figure 2. The model results in a test loss of 1.03 (see Table 5).

2.2.3 Multi-head model

To find the correct time corresponding to the image, we also implemented a multi-head model. Instead of one output, the model has an output for the hour and the minutes of the time. The hour output is done through classification and the minutes output uses regression. The hours are simply labeled by taking the hour value of the time and the minutes by dividing by 60. The implementation of the model can be found in Table 15.

Implementing such a model becomes more tricky because we are now trying to implement both a regression and classification feature at the same time. We noticed that dense layers cause overfitting on classifications, but were important for regressive models. Therefore, we added an extra layer for the regressive output, which is not connected to the classification output.

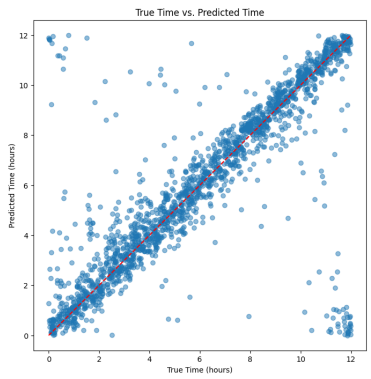
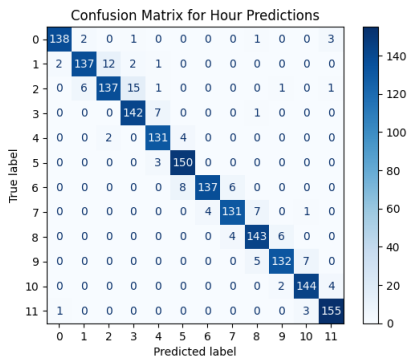
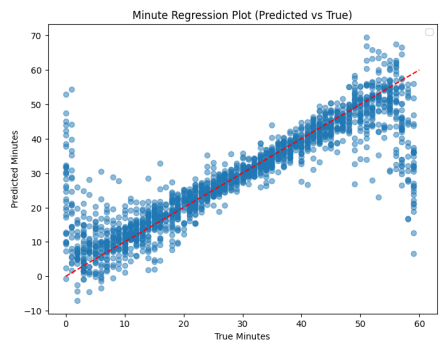


Figure 2: The distribution of the true and predicted values of the regression model on the 75x75 set.



(a) Prediction of hours of multi-head model.



(b) Prediction of minutes of multi-head model.

Figure 3: Output of multi-head model.

When looking at Figure 3, we can see that the model predicts quite well with 30 epochs of training. The test set accuracy of the classification output in this model is 93.2%, and the MAE (Mean Absolute Error) of the regressive output is 0.077. This means that the average error is approximately only 5 minutes ($0.077 \times 60 = 4.62$ minutes).

2.3 Final evaluation

Finally, the best-performing models from the aforementioned implementations will be tested on the 150x150 dataset. An overall worse performance is expected on each model. Firstly, because larger images lead to more parameters in the network, increasing the likelihood of overfitting. Besides, CNNs are particularly good at capturing spatial features through a series of convolutional layers. With smaller images, CNNs can focus on learning these core features without needing as many layers or

filters. Larger images, on the other hand, may require deeper networks to capture fine-grained details, increasing complexity.

2.3.1 Classification

The classifier model on the 150x150 is observed. In Table 4 the accuracies are presented. The accuracy all worsened because the model is relatively small and not able to gain important information to make correct predictions. The dropoff was less extreme, which could be because there is more detail. Increasing the number of neurons in the convolutional layers and increasing the pool size of the Max Pooling layers would improve the accuracy, but it is interesting to observe the changes in accuracy when changing the input from 75x75 to 150x150 with the same model. The training accuracy remains the same regardless of the size of the input images, but the test and validation accuracy is lower for the 150x150 images. This means that there is more overfitting. Classification with 720 categories is simply impossible with this model and results in a test accuracy of 0.00%.

Categories	Accuracy
24	58.56%
72	53.56%
144	45.11%
720	0.00%

Table 4: Accuracy on 150x150 per number of categories.

2.3.2 Single-head regression

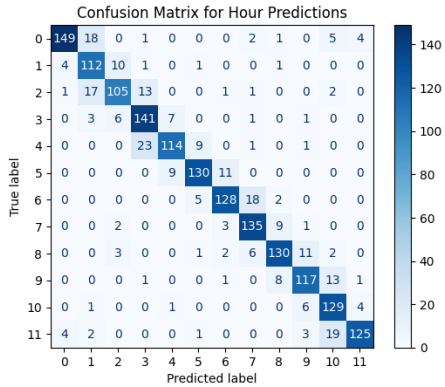
When testing the earlier defined model for the 75x75 input files on the 150x150 input files we see a lot of overfitting as the loss on the train set is below 0.5, while the actual evaluation on the test set is around 3. When experimenting with different settings we found that increasing the pooling layer size decreases the overfitting and results in a better loss for the test set, as seen in Table 5.

Input size	Pooling layer size	Train loss	Test Loss
75x75	(5,5)	0.7594	1.0320
150x150	(5,5)	1.0915	2.1960
150x150	(13,13)	1.3328	1.3882

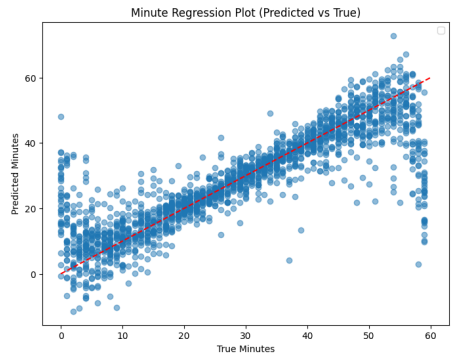
Table 5: Loss for the test function for different models and pooling layer sizes.

2.3.3 Multi-head model

When looking at Figure 4 we can see that, much like on the 75x75 dataset, the same model can converge to a suitable solution. The minute loss is 0.089 and the validation accuracy of the classifier is 85.2% after 30 epochs.



(a) Prediction of hours of multi-head model.



(b) Prediction of minutes of multi-head model.

Figure 4: Output of multi-head model on 150x150.

Surprisingly, unlike for single-head regression, increasing the pool size to (5,5) or (13,13) worsened the performance of the model in both classification and regression.

2.4 Conclusion

In conclusion, while comparing the models we found that the different approaches to predict the time and minimize the common sense error have their advantages and disadvantages. While training a classification model to predict the time in 24 intervals is quite successful and fast to train, increasing the number of intervals quickly reduces the accuracy. A regression model, with a custom loss function that accounts for the common sense error, can be advantageous since its time prediction is not bound to a discrete number of categories. However, the regression loss could not be reduced to less than an hour, whereas the classification labeled 83% of the test images in the correct half-hour interval. Comparing them, they both often result in 1 hour errors. The model that is best at generalizing the data is the model that combines these two concepts, the multi-head model. This model used the high-accuracy category prediction of the classification model for the hours and then performed regression to predict the minutes. This results in a model that gets very close to the actual time.

When we compare the results between the 75x75 and 150x150 datasets we see that the models have worse results on the larger dataset. At first, we expected the models to perform better on the larger datasets because it has more information to learn from, but our models are too small and have insufficient budget to correctly handle these larger images.

While training our models we found that for every model the convergence was initially very fast. This would later turn into one of our main issues, as the models would often overfit. This could have been improved by increasing the model complexity, but we used Google Colab and would often run into limitations due to the limited amount of RAM available. Another method that could be implemented to limit overfitting is regularization.

References

[AAB+15] Martín Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, Zhifeng Chen, Craig Citro, Greg S. Corrado, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Ian Goodfellow, Andrew Harp, Geoffrey Irving, Michael Isard, Yangqing Jia, Rafal Jozefowicz, Lukasz Kaiser, Manjunath Kudlur, Josh Levenberg, Dandelion Mané, Rajat Monga, Sherry Moore, Derek Murray, Chris Olah, Mike Schuster, Jonathon Shlens, Benoit Steiner, Ilya Sutskever, Kunal Talwar, Paul Tucker, Vincent Vanhoucke, Vijay Vasudevan, Fernanda Viégas, Oriol Vinyals, Pete Warden, Martin Wattenberg, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. TensorFlow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org.

[C+15] François Chollet et al. Keras. `https://keras.io`, 2015.

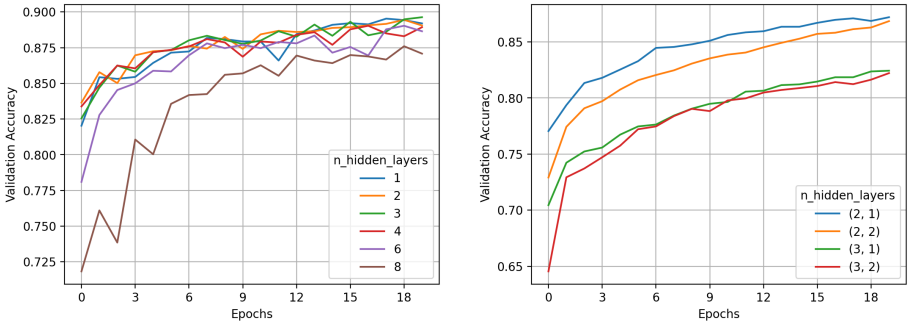
[Ge7] Aurelien Geron. *Hands-on machine learning with Scikit-Learn and TensorFlow : concepts, tools, and techniques to build intelligent systems.* O'Reilly Media, Sebastopol, CA, 2017.

[Kri09] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical Report TR-2009, University of Toronto, 2009.

[ML18] Dominic Masters and Carlo Luschi. Revisiting small batch training for deep neural networks. *CoRR*, abs/1804.07612, 2018.

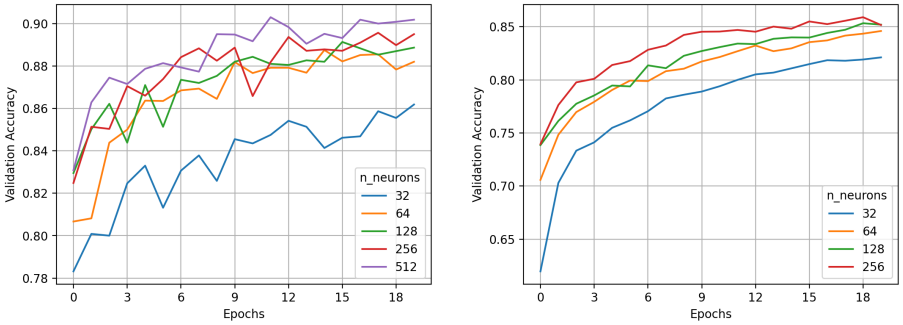
[XRV17] Han Xiao, Kashif Rasul, and Roland Vollgraf. Fashion-mnist: a novel image dataset for benchmarking machine learning algorithms. *arXiv preprint arXiv:1708.07747*, 2017.

A Task 1 Figures & Tables



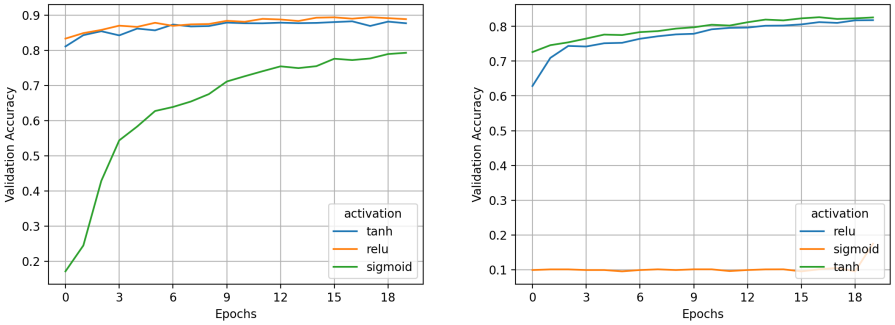
(a) MLP ***(b)*** CNN

Figure 5: Different number of hidden layers



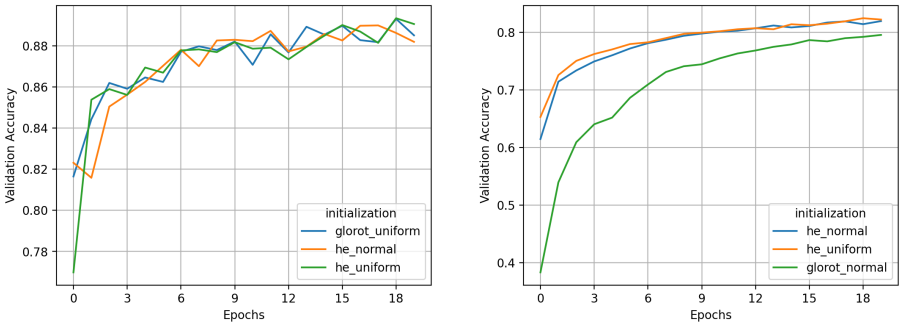
(a) MLP ***(b)*** CNN

Figure 6: Different number of neurons



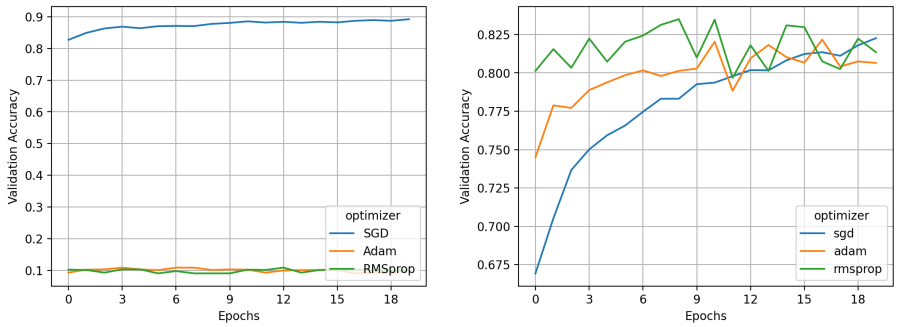
(a) MLP ***(b)*** CNN

Figure 7: Different activations functions



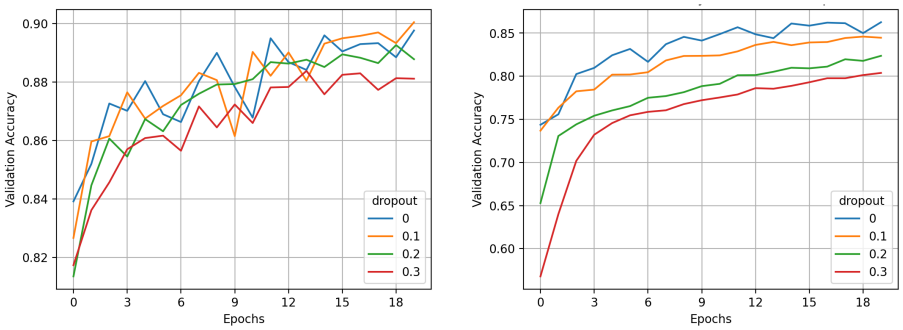
(a) MLP ***(b)*** CNN

Figure 8: Different initializations



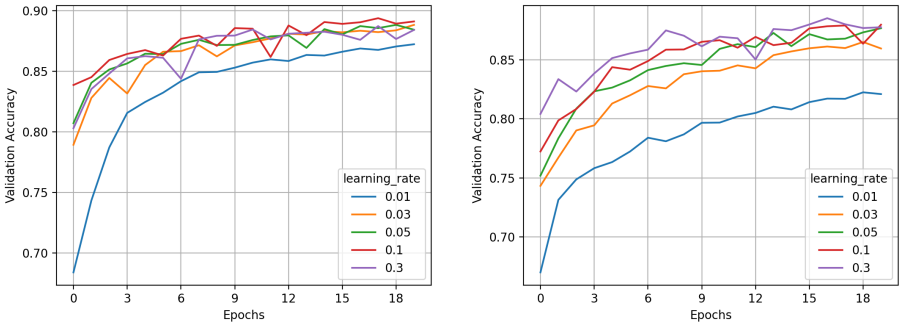
(a) MLP ***(b)*** CNN

Figure 9: Different optimizers



(a) MLP ***(b)*** CNN

Figure 10: Different dropout values



(a) MLP ***(b)*** CNN

Figure 11: Different learning rates

Table 6: Test set loss and accuracy for different numbers of hidden layers for an MLP and CNN algorithm

	MLP	
	Loss	Accuracy
1	0.3434	0.8782
2	0.339	0.8767
3	0.3394	0.8768
4	0.3509	0.8793
6	0.3706	0.8711
8	0.4116	0.8553

	CNN	
	Loss	Accuracy
(2,1)	0.3828	0.8627
(2,2)	0.3951	0.8552
(3,1)	0.5161	0.8155
(3,2)	0.5137	0.8117

Table 7: Test set loss and accuracy for different weight initialization methods for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
Glorot uniform	0.3562	0.8729	0.5857	0.7847
He normal	0.3654	0.8697	0.5033	0.8128
He uniform	0.3477	0.8752	0.51	0.8119

Table 8: Test set loss and accuracy for different numbers of neurons for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
32	0.4642	0.8408	0.5223	0.8134
64	0.3860	0.8599	0.4430	0.8390
128	0.3509	0.8754	0.4182	0.8517
256	0.3500	0.8742	0.4097	0.8484
512	0.3178	0.8899	-	-

Table 9: Test set loss and accuracy for different activation functions for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
tanh	0.3837	0.8632	0.506	0.8201
reLU	0.3547	0.8737	0.5186	0.8098
sigmoid	0.6243	0.773	2.3015	0.1721

Table 10: Test set loss and accuracy for different dropout values for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
0	0.3312	0.885	0.3919	0.8606
0.1	0.332	0.8803	0.4352	0.8417
0.2	0.3592	0.8697	0.5035	0.8146
0.3	0.3668	0.8701	0.5738	0.7920

Table 11: Test set loss and accuracy for different optimizers for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
SGD	0.3431	0.8786	0.5164	0.8137
Adam	2.3055	0.1	0.5323	0.7976
RMSprop	2.3106	0.1	0.5360	0.8084

Table 12: Test set loss and accuracy for different learning rates for an MLP and CNN algorithm.

	MLP		**CNN**	
	Loss	Accuracy	Loss	Accuracy
0.01	0.4021	0.8547	0.5059	0.8120
0.03	0.3579	0.8747	0.3899	0.8541
0.05	0.3559	0.8700	0.3500	0.8700
0.1	0.3475	0.8762	0.3356	0.8735
0.3	0.3611	0.8658	0.3435	0.8681

B Task 2 Tables

Layer	Settings
Conv2D	16, kernel size(3, 3), relu
Conv2D	32, kernel size(3, 3), relu
MaxPooling2D	pool size(5,5)
Dropout	20%
Flatten	-
Dense	1024, relu
Dropout	10%
Dense	512, relu
Dense	#categories, softmax

Table 13: Classification model used for the final evaluation.

Layer	Settings
Conv2D	16, kernel size(3, 3), relu
Conv2D	32, kernel size(3, 3), relu
MaxPooling2D	pool size(5,5) or (13,13)
Dropout	20%
Flatten	-
Dense	1024, relu
Dropout	10%
Dense	512, relu
Dense	1, linear

Table 14: Regression model used for the final evaluation.

Layer	Settings
Conv2D	16, kernel size(3, 3), relu
MaxPooling2D	pool size(2,2)
Conv2D	32, kernel size(3, 3), relu
MaxPooling2D	pool size(2,2)
Conv2D	64, kernel size(3, 3), relu
MaxPooling2D	pool size(2,2)
Dropout	20%
Flatten	-
Dense	1024, relu
Dropout	10%
Dense (Classifier output)	12, softmax
Dense	1024, relu
Dense (Regression output)	1, linear

Table 15: Multi-head model used for the final evaluation.